

The Vertex Model in Code

Where the DAG and the UTXO model become Python classes — GenericVertex, Block, Transaction, their inputs, outputs, and the metadata the node maintains about each.

SUBJECT hathor-core · Part II · the node, end to end

CHAPTER 25 · Part II · The Node

BRANCH study/hathor-core-studies

EDITION 2026 · v1

GenericVertex · Block · Transaction · TxInput · TxOutput · parents vs inputs · metadata · static metadata · tokens · validation state

Table of Contents

Chapter 25 — The Vertex Model in Code	3
25.1 Localization	4
25.2 What it does and why it exists	4
25.3 The concepts it rests on	5
25.4 A hand-traced example first	6
25.5 The code, walked	7
25.5.1 GenericVertex: the abstract generic base	7
25.5.2 The shared fields, and what each one models	8
25.5.3 TxInput and TxOutput: the UTXO model, literally	10
25.5.4 Parents versus inputs, in the fields	12
25.5.5 The subclass hierarchy	12
25.5.6 Hashing and serialization (pointer forward)	14
25.6 The metadata split: static versus mutable	14
25.7 Validation state and storage scope	17
25.8 How it plugs into the lifecycle	18
Recap	18

Chapter 25 — The Vertex Model in Code

What you'll learn in this chapter

- How the abstract ideas of Chapters 7 and 8 — the UTXO model and the DAG of vertices — become concrete Python classes in `hathor/transaction/`.
- The shape of the **GenericVertex** base class: the fields every block and every transaction share, what each field models in the real world, and why the base is generic and abstract.
- The two concrete subclasses — **Block** and **Transaction** — and the further specializations built on them (`MergeMinedBlock`, `PoaBlock`, `TokenCreationTransaction`, `OnChainBlueprint`).
- The **TxInput** and **TxOutput** classes in code, mapped one-to-one onto the UTXO concepts of Chapter 7, including how a single byte encodes which token an output holds and whether it is an authority.
- The crucial **parents-versus-inputs** distinction (Chapter 8) seen directly in the fields, and how the code keeps the two edge sets separate.
- The **metadata split** — immutable static metadata versus the mutable `TransactionMetadata` that consensus rewrites — and why a node needs both.
- Where **validation state** and storage **scope** fit, and how a vertex travels through the node's lifecycle.

Two earlier chapters built the theory this chapter pays off. Chapter 7 taught the **UTXO model**: money as a heap of discrete, individually-locked outputs, where a transaction consumes some and creates others. Chapter 8 taught the **DAG**: a ledger arranged as a directed acyclic graph of vertices, where each vertex has two distinct kinds of outgoing edge — **parents** (the confirmation/topology edge) and **inputs** (the spending edge). This chapter does not re-teach either; it shows you the real classes that are those ideas. By the end you should be able to open `base_transaction.py` and read it as fluently as the diagrams in Chapters 7 and 8.

Everything here lives in one package, `hathor/transaction/`, and overwhelmingly in one file, `base_transaction.py` (about 1150 lines). We start by locating the package, then state what it is for, recap the concepts it rests on, and finally walk the code in detail.

25.1 Localization

The package sits in the **domain-model** group of the module map (Chapter 0, §0.4). It is foundational: storage, verification, consensus, indexes, p2p, and the wallet all import these classes. Nothing in the node manipulates a block or a transaction without going through the types defined here.

```
hathor/
├── transaction/
│   ├── base_transaction.py    ◀ YOU ARE HERE – GenericVertex, TxInput, TxOutput, TxVersion
│   ├── block.py              ◀ Block (subclass of GenericVertex[BlockStaticMetadata])
│   ├── transaction.py        ◀ Transaction (subclass of GenericVertex[TransactionStaticMet
│   ├── merge_mined_block.py  ◀ MergeMinedBlock (subclass of Block)
│   ├── token_creation_tx.py   ◀ TokenCreationTransaction (subclass of Transaction)
│   ├── static_metadata.py     ◀ VertexStaticMetadata / BlockStaticMetadata / TransactionSta
│   ├── transaction_metadata.py ◀ TransactionMetadata (the mutable, computed state)
│   ├── validation_state.py    ◀ ValidationState enum (INITIAL → BASIC → FULL ...)
│   ├── genesis.py            ◀ is_genesis(), the graph's root
│   ├── types.py              ◀ TokenInfo for token accounting
│   └── aux_pow.py            ◀ Bitcoin auxiliary proof-of-work (for merge mining)
├── scripts/                  ◀ the locking/unlocking script language (P2PKH, opcodes) → Ch
├── headers/                  ◀ optional vertex headers (nano-contract, fee)
├── poa/                      ◀ PoaBlock (proof-of-authority block) → Ch 32
└── storage/                  ◀ vertex-aware storage + graph-walk utilities → Ch 27
```

Context. This package is the node's **data model** — the in-memory and on-disk shape of every block and transaction in the ledger. It defines what a vertex is (its fields), what kinds of vertex exist (the class hierarchy), and what the node remembers about each one (the two metadata objects). It does **not** decide whether a vertex is valid — that is verification, Chapter 31 — nor which vertices win a conflict — that is consensus, Chapter 32. It is the vocabulary those chapters speak.

25.2 What it does and why it exists

A full node's entire job is to hold a ledger and reason about it. Before it can store a transaction, verify it, or sync it to a peer, it needs an answer to a flat-footed question: what is a transaction, as a thing in memory? This package answers that. It gives the node:

1. **A representation for every ledger object.** A class whose instances hold the data a block or transaction is made of — its outputs, its inputs (if any), the vertices it confirms, its proof-of-work nonce and weight.
2. **One umbrella type plus specific kinds.** A common base (`GenericVertex`) so the rest of the node can pass around "a vertex" without caring whether it

holds a block or a transaction, and concrete subclasses (`Block`, `Transaction`, and four more) for the cases where the kind does matter.

3. **A precise map onto the domain models.** The `TxInput` / `TxOutput` classes are the UTXO model of Chapter 7 made literal; the `parents` field is the DAG edge of Chapter 8 made literal.
4. **A place to keep what the node computes about each vertex.** A vertex's own contents are fixed once it is created (you cannot un-sign a transaction). But the node learns facts about a vertex over time — how much accumulated work stands behind it, whether it has been voided, which block first confirmed it. That derived knowledge lives in **metadata** objects attached to the vertex, kept separate from the vertex's own immutable data. The separation is the most conceptually interesting thing in the package, and §25.6 is devoted to it.

Why does this deserve its own large package rather than, say, a couple of dataclasses? Because a vertex is not a passive bag of fields. It knows how to hash itself (the proof-of-work identity), serialize itself to the wire format, compute its dependencies, and answer questions like "what addresses are involved here?" The class gathers the data and the operations that belong with it — the object-oriented principle of Chapter 1 (encapsulation: keep the data and the code that maintains its invariants together).

25.3 The concepts it rests on

Four ideas from earlier chapters meet here. Each gets a recap box; the full treatment is in the cited chapter.

Recap — vertex, DAG, parents (full treatment in Ch. 8). Hathor's ledger is one directed acyclic graph. Its nodes are called **vertices**; a vertex is either a **block** or a **transaction**. Each vertex names a short list of **parents** — earlier vertices it attaches to and thereby confirms. Parents are the structural (topology) edge; they are the generalization of a classic blockchain's single "previous block" pointer. A block has three parents (one block-parent continuing the block backbone, plus two transaction-parents); a transaction typically has two. → Ch 8.

Recap — UTXO, inputs, outputs, scripts (full treatment in Ch. 7). Money is recorded as a set of discrete **unspent transaction outputs** (UTXOs), each holding a value and a locking script that says who may spend it. A transaction **consumes** existing outputs (its **inputs**, each a pointer `(tx_id, index)` to a prior output) and **creates** new ones. There is no stored balance and no owner field; ownership is "can you satisfy the locking script?" Inputs and outputs are the spending edge — entirely separate from parents. → Ch 7.

Recap — inheritance, abstract base classes, generics (full treatment in Ch. 1 and Ch. 5). A **subclass** inherits fields and methods from a base class and may add or override its own. An **abstract base class** (ABC) defines a shared interface but cannot itself be instantiated; it marks some methods `@abstractmethod`, forcing every concrete subclass to supply them. A **generic** class is parameterized by a type variable (written `Class[T]`), so the same class definition can be specialized to different companion types while keeping them straight for the type-checker. `GenericVertex[StaticMetadataT]` uses all three at once. → Ch 1, Ch 5.

Recap — encapsulated invariants (full treatment in Ch. 1). A well-designed class does not merely store fields; it guards rules about them ("a vertex's hash, once set, must match its contents", "outputs may only be added, never mutated"). The class keeps the data and the code that maintains those rules together, so no outside code can put an object into an inconsistent state. Watch for these invariants as we walk the code; they are the why behind otherwise puzzling design choices. → Ch 1.

25.4 A hand-traced example first

Before the real classes, build the picture with plain objects. Suppose Alice holds two unspent outputs — a 50-HTR output and a 20-HTR output, both created by some earlier transaction `X` — and wants to pay Bob 30 HTR. From Chapter 7 we know this is one transaction that consumes both inputs and creates two outputs (one to Bob, one of change back to Alice). From Chapter 8 we know the same transaction must also name **parents** to attach itself to the DAG.

Written as Python-shaped pseudocode (not yet the real API), the transaction is two independent lists plus a parents list:

```
tx = Transaction(
    # --- the SPENDING edge (UTXO model, Ch 7) ---
    inputs=[
        TxInput(tx_id=X_hash, index=0, data=<Alice's signature + pubkey>), # the 50-HTR output
        TxInput(tx_id=X_hash, index=1, data=<Alice's signature + pubkey>), # the 20-HTR output
    ],
    outputs=[
        TxOutput(value=30, script=<lock to Bob>),      # Bob's coin
        TxOutput(value=40, script=<lock to Alice>),    # Alice's change
    ],
    # --- the TOPOLOGY edge (DAG, Ch 8) — UNRELATED to the above ---
    parents=[parent_hash_1, parent_hash_2],          # two tips this tx confirms
)
```

Three things to notice, because each becomes a real design point:

- **inputs** and **parents** are different lists with different meanings. The inputs say "I spend these coins"; the parents say "I confirm these vertices." A

tx Alice spends from (X) need not be among her parents, and her parents need not be txs she spends from. Conflating them is the classic beginner error Chapter 8 warned about.

- **Conservation is not enforced by the constructor.** Inputs sum to 70, outputs to 70 — but the class does not check that. Constructing the object and validating it are separate steps (validation is Chapter 31). The class's job is to hold the data faithfully; the rules live elsewhere. (This is itself a design decision we will see honored throughout.)
- **There is no from field and no balance.** Ownership of the consumed coins is proved by the `data` (unlocking script) inside each input, exactly as Chapter 7 described.

Now the real classes.

25.5 The code, walked

25.5.1 `GenericVertex` : the abstract generic base

The umbrella type is `GenericVertex`, declared at `base_transaction.py:148`:

```
StaticMetadataT = TypeVar('StaticMetadataT', bound=VertexStaticMetadata, covariant=True)

class GenericVertex(ABC, Generic[StaticMetadataT]):
    """Hathor generic vertex"""
```

Three things are happening in that one line, each from a recap box above:

- **ABC** — it is an abstract base class. You cannot write `GenericVertex(...)`; you must instantiate a concrete subclass (`Block` or `Transaction`). The base marks several methods `@abstractmethod` — for example `is_block` (`base_transaction.py:255`), `is_transaction` (`:260`), `get_funds_struct` (`:438`), and `get_token_uid` (`:895`) — so every concrete vertex must answer them. The base provides everything common; the subclasses fill in what differs.
- **`Generic[StaticMetadataT]`** — it is generic over a type variable. The companion "static metadata" type differs per vertex kind (a block's static metadata records its height; a transaction's does not). Rather than have one untyped metadata field, the base is parameterized so that `Block` is `GenericVertex[BlockStaticMetadata]` and `Transaction` is `GenericVertex[TransactionStaticMetadata]`. The type-checker then knows that `some_block.static_metadata.height` is valid but `some_tx.static_metadata.height` is not. (`bound=VertexStaticMetadata` constrains the variable: whatever you plug in must be a subclass of `VertexStaticMetadata`. This is the generics machinery of Chapter 5.)

- `StaticMetadataT` is the name of that type variable. We meet the static-metadata classes in §25.6.

Aside — why "GenericVertex" and not just "Vertex"? The names `Vertex` and `BaseTransaction` exist, but as aliases, defined just after the class (`base_transaction.py:932`):

```
> Vertex: TypeAlias = GenericVertex[VertexStaticMetadata]
> BaseTransaction: TypeAlias = Vertex
>
```

`Vertex` is `GenericVertex` with its type parameter pinned to the common base metadata — the type you use when you hold "some vertex, kind unknown."
`BaseTransaction` is kept only for backwards compatibility; the docstring at `:927` says it "can be removed in the future." So when you see `BaseTransaction` in older code, read it as "any vertex." Throughout the rest of the codebase, function signatures say `Vertex` or `BaseTransaction` and mean exactly the same thing.

25.5.2 The shared fields, and what each one models

Every vertex is built by the base `__init__` (`base_transaction.py:170`). The fields are listed in `__slots__` at `:151`; here is what each one models in the world and why it must exist. (`__slots__` is a memory optimization: it tells Python the exact, fixed set of attributes an instance may have, so each object can skip its per-instance dictionary. With millions of vertices in memory this matters; it also doubles as a guard against typo-ing a new attribute into existence.)

Field	Set at	Models	Why it must exist
<code>version</code>	<code>:200</code>	which kind of vertex this is (a <code>TxVersion</code> enum)	the wire format and the dispatcher use it to pick the right class to parse the bytes (<code>TxVersion.get_cls</code> , <code>:115</code>)
<code>signal_bits</code>	<code>:199</code>	a byte of extra bits a block uses for feature-activation miner signalling (Ch 38)	lets a block carry "I vote for upgrade X" without a new field; ignored by transactions
<code>weight</code>	<code>:201</code>	the proof-of-work effort of this vertex, as <code>log2(work)</code> (Ch 9)	the unit consensus sums into accumulated weight; anti-spam for transactions

Field	Set at	Models	Why it must exist
timestamp	:198	the Unix time the vertex was created	ordering, tie-breaking conflicts (Ch 10), and validity windows
nonce	:197	the number a miner varied to make the hash meet the target (Ch 9)	the proof-of-work solution; part of what is hashed
inputs	:202	the UTXOs this vertex spends — a <code>list[TxInput]</code> (Ch 7)	the spending edge; empty for blocks
outputs	:203	the new coins this vertex creates — a <code>list[TxOutput]</code> (Ch 7)	where value (and block reward) comes from
parents	:204	the vertices this one confirms — a <code>list[VertexId]</code> (Ch 8)	the DAG topology edge; the graph itself
_hash	:206	this vertex's identity: the double-SHA256 of its contents	the key it is stored under; what parents and inputs point at
storage	:205	a back-reference to the <code>TransactionStorage</code> (or <code>None</code>)	lets a vertex look up its parents and spent outputs on demand (Ch 27)
_metadata	(lazy)	the mutable, computed <code>TransactionMetadata</code> (§25.6)	the node's evolving knowledge about the vertex
_static_metadata	:207	the immutable computed <code>VertexStaticMetadata</code> (§25.6)	derived facts that never change (e.g. a block's height)
headers	:209	optional trailing sections (nano-contract header, fee header)	extensibility without changing the core format (Ch 39)
name	:212	a debug-only label	convenience in tests and logs; never serialized

A few of these reward a closer look.

`_hash` and the identity invariant. A vertex is identified by its hash, and the hash is computed from its contents. This is the central invariant of the class: **the stored `_hash` must equal `calculate_hash()` of the current fields**. The hash is the double-SHA256 of the serialized vertex (`calculate_hash` , :628 , built from `calculate_hash1` / `calculate_hash2` at :603 / :613 , which fold in the nonce at :624). Because the hash depends on every field, changing any field would change the identity — which is exactly why the outputs and inputs must be treated as immutable once the vertex exists. The leading underscore on `_hash` and the property guard at :340 (which asserts the hash has been set before anyone reads it) enforce that you never hand out an uninitialized identity.

`parents` is a list of `VertexId` , not of vertices. `VertexId` is just an alias for `bytes` (`types.py:26`) — a 32-byte hash. The vertex stores the hashes of its parents, not the parent objects. To get the actual parent objects you call `get_parents()` (:382), which asks the storage to fetch each one by hash. This keeps a vertex small and avoids loading the whole graph into memory at once; it is why `storage` is a field. The same is true of `TxInput.tx_id` — inputs point at prior transactions by hash, not by reference.

`inputs` is empty for blocks. Blocks mint coins; they do not spend them (Chapter 7, §7.3). The `Block.__init__` (`block.py:48`) does not even accept an `inputs` argument — it calls `super().__init__` without one, leaving the list empty. A transaction's `__init__` (`transaction.py:60`) accepts inputs and, additionally, a `tokens` list (next section).

25.5.3 `TxInput` and `TxOutput` : the UTXO model, literally

These two small classes are Chapter 7 turned into code. `TxInput` (`base_transaction.py:936`) is the spending pointer:

```
class TxInput:
    def __init__(self, tx_id: VertexId, index: int, data: bytes) -> None:
        # tx_id: hash of the transaction that contains the output of this input
        # index: index of the output you are spending from transaction tx_id
        # data: data to solve output script (the unlocking script)
        self.tx_id = tx_id
        self.index = index
        self.data = data
```

The `(tx_id, index)` pair is the "pointer to a specific earlier output" from §7.3: "spend output number `index` of transaction `tx_id` ." The `data` field is the **unlocking script** of §7.5 — the bytes (typically a signature and public key) that satisfy the spent output's locking script. Note `get_sighash_bytes` (:971): when a transaction is hashed for signing, each input's `data` is cleared to zero-length, because you cannot sign over the very signature you are about to produce. That is a subtle correctness detail the class hides from its callers.

`TxOutput` (`base_transaction.py:1022`) is a coin: a value plus a lock plus a token tag.

```

class TxOutput:
    TOKEN_INDEX_MASK      = 0b01111111  # low 7 bits: which token
    TOKEN_AUTHORITY_MASK = 0b10000000  # high bit:  is this an authority output?

    def __init__(self, value: int, script: TxOutputScript, token_data: int = 0) -> None:
        if value <= 0 or value > MAX_OUTPUT_VALUE:
            raise InvalidOutputValue
        self.value = value          # amount (or, for authorities, an authority bitmask)
        self.script = script        # the locking script (Ch 7 §7.5)
        self.token_data = token_data

```

`value` and `script` are exactly the "value" and "locking script" of Chapter 7. The constructor does enforce one invariant — `value` must be positive and within range (: 1044) — but, as promised in §25.4, it does **not** check conservation against any inputs; that is verification's job.

The interesting field is `token_data`, a single byte that does two jobs through bit-masking. Recall from Chapter 7 that Hathor is multi-token: one output might hold HTR, another a custom token. Rather than spend a whole separate field, the byte packs both "which token" and "is this special" together:

- **The low 7 bits — which token.** `get_token_index()` (: 1099) returns `token_data & TOKEN_INDEX_MASK`. Index 0 always means HTR, the native token (whose UID is the single byte `b'\x00'`, `conf/settings.py:29`); higher indices point into the transaction's `tokens` list. This is why a transaction carries a `tokens` list and a block does not — a block only ever holds HTR (`Block.get_token_uid` asserts `index == 0`, `block.py:256`).
- **The high bit — authority or value.** `is_token_authority()` (: 1103) returns true when `token_data & TOKEN_AUTHORITY_MASK` is set. An **authority output** carries no spendable money; instead its `value` field is reinterpreted as a permissions bitmask — bit 0 is the right to **mint** more of the token, bit 1 the right to **melt** (destroy) it (`TOKEN_MINT_MASK / TOKEN_MELT_MASK`, : 1029 – : 1031 ; checked by `can_mint_token / can_melt_token`, : 1115 / : 1119). This is the "a coin that grants a power rather than holding a value" idea from §7.6, made concrete: the same `TxOutput` class expresses both money and authority, distinguished by one bit.

A hand-trace of the byte: `token_data = 0b10000001` means "high bit set → authority output" and "low bits 0000001 → token index 1." Combined with a `value` of `0b01` (`TOKEN_MINT_MASK`), it is "a mint-authority output for the transaction's first custom token." The bookkeeping that consumes these — checking that an output's mint authority was also present on some input, so authorities cannot be conjured — lives in `Transaction._update_token_info_from_outputs` (`transaction.py:432`), pointing forward to verification (Ch 31).

25.5.4 Parents versus inputs, in the fields

Chapter 8 §8.4 called the parents/inputs distinction "the single most consequential distinction in the book." Here it is, sitting in the class as two separate fields with two separate types:

```
self.inputs = inputs or [] # list[TxInput] - the SPENDING edge (Ch 7)
self.parents = parents or [] # list[VertexId] - the TOPOLOGY edge (Ch 8)
```

The code keeps them rigorously apart, and two methods make the separation visible:

- `get_all_dependencies()` (`base_transaction.py:487`) unions both edge sets:

```
return set(chain(self.parents, (i.tx_id for i in self.inputs)))
```

It exists because to process a vertex you need everything it points at, either way — you cannot verify a transaction without both its parents and the transactions it spends from. But notice it builds one set only to walk dependencies; it never merges the roles.

- `get_tx_parents_ids()` (`:491`) deals with the block's asymmetry:

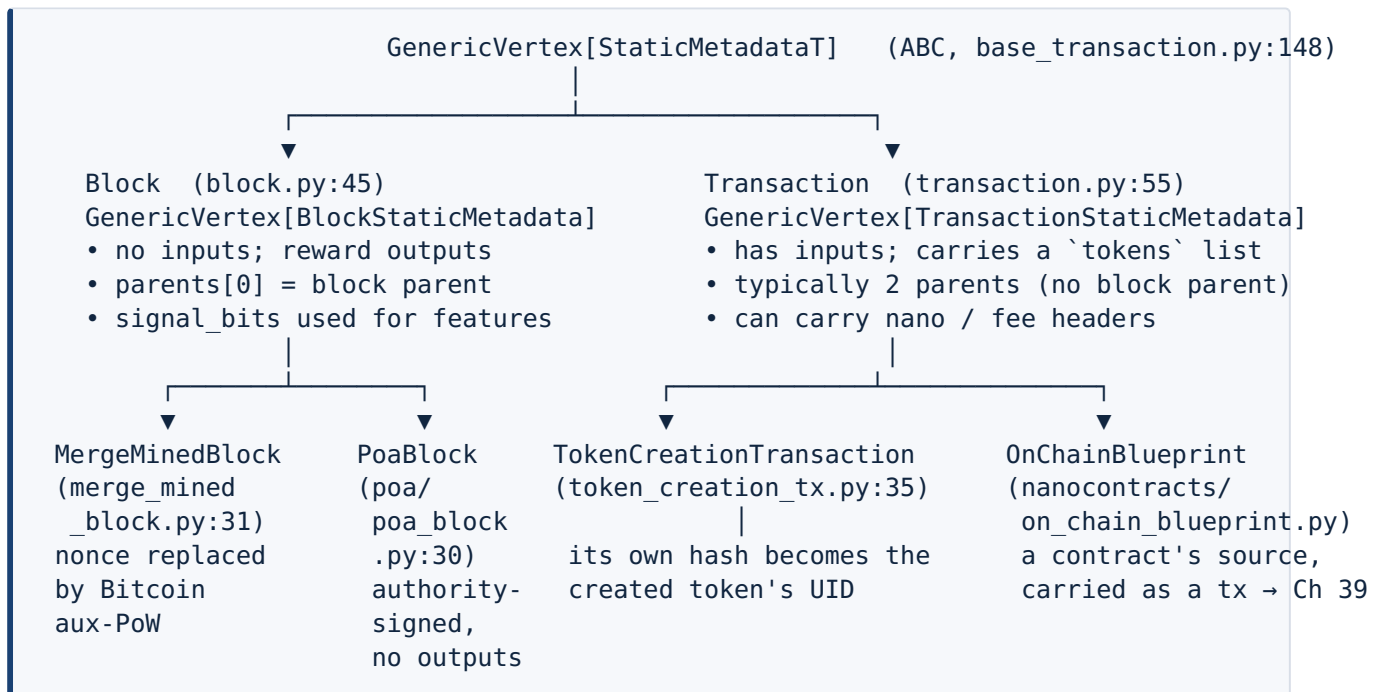
```
return set(self.parents[1:] if self.is_block else self.parents)
```

For a block, `parents[0]` is the block-parent (the block-backbone edge); the rest are transaction-parents. So this method drops `parents[0]` for blocks and keeps all parents for transactions. The `Block` class exposes the dropped one directly via `get_block_parent_hash()` → `return self.parents[0]` (`block.py:169`). This is exactly the "a block has three parents, `parents[0]` is the block parent" claim from Chapter 8, now visible in code.

The payoff: a **double-spend** is a collision in the input edges (two transactions whose `inputs` cite the same `(tx_id, index)`), and the class even has a method to detect it on the fly — `Transaction.is_double_spending()` (`transaction.py:463`) walks its inputs and asks each spent output's metadata whether it is already spent by someone else. The graph (parents) happily holds both conflicting transactions; resolving the conflict is consensus (Ch 32). The structure contains conflicts; it does not prevent them — precisely as Chapter 8 promised.

25.5.5 The subclass hierarchy

`GenericVertex` is abstract; the real objects are its descendants. There are two concrete branches and four leaf specializations.



What distinguishes each subclass, and why:

- **Block** (block.py:45) declares its kind via `is_block → True / is_transaction → False` (:82 / :87). It overrides the funds serialization to have no inputs and no token list (`get_funds_struct`, :222), adds a small `data` field (:74), and exposes the block-parent helpers (`get_block_parent_hash`, :166). Its `get_token_uid` asserts index 0 and returns HTR (:245) — blocks are HTR-only.
- **Transaction** (transaction.py:55) is the mirror: `is_transaction → True`, has inputs, and adds a `tokens` list in its own `__slots__` (:56) — the list of custom-token UIDs that an output's `token_data` index points into (`get_token_uid`, :277). It also caches the sighash (:93) because signing a multi-input transaction re-serializes the same body many times.
- **MergeMinedBlock** (merge_mined_block.py:31) extends **Block** for merged mining (Chapter 0; full treatment Ch 37). A miner mines Hathor as a side-effect of mining Bitcoin, so the proof-of-work is a Bitcoin "auxiliary PoW" object rather than a plain nonce. The subclass swaps the nonce out (`get_struct_nonce` returns the aux-PoW bytes, :84) and computes its hash through that aux-PoW (`calculate_hash`, :80). Everything else it inherits.
- **PoaBlock** (poa/poa_block.py:30) extends **Block** for proof-of-authority networks (private chains; Ch 32). It **asserts it has no outputs** (:47) — PoA blocks carry no reward — and replaces proof-of-work with a `signer_id` and a `signature` (:61), serialized into the graph fields (`get_graph_struct`, :86). Authority, not work, is what makes the block valid.
- **TokenCreationTransaction** (token_creation_tx.py:35) extends **Transaction** to create a custom token. Its defining trick: **its own hash becomes the new token's UID**. `update_hash()` is overridden so that after the hash is computed,

`self.tokens = [self.hash] (:86 - :90)` — the transaction declares the token it just minted. It also carries the token's name, symbol, and version.

- **OnChainBlueprint** extends `Transaction` to carry a nano-contract blueprint (contract source) on the ledger. It is only registered when nano-contracts are enabled (`TxVersion.get_cls`, `base_transaction.py:131`), and its internals belong to the nano-contracts subsystem — Chapter 39.

How does a stream of bytes off the wire become the right one of these classes? The `version` field. `TxVersion` (`base_transaction.py:99`) is an integer enum (`REGULAR_BLOCK = 0`, `REGULAR_TRANSACTION = 1`, `TOKEN_CREATION_TRANSACTION = 2`, ...), and `TxVersion.get_cls()` (:115) is a small dispatch table mapping each version to its class. The parser reads the version byte, looks up the class, and asks that class to build itself from the bytes. This is the factory-by-dispatch pattern of Chapter 3, and it is why every vertex carries its own version.

25.5.6 Hashing and serialization (pointer forward)

A vertex's bytes are produced in layers — `get_funds_struct()` (the value part: inputs, outputs, tokens), `get_graph_struct()` (the topology part: weight, timestamp, parents), then the nonce and any headers — assembled by `get_struct()` (:477). The hash is the double-SHA256 over a fixed 64-byte mining header (`get_mining_header_without_nonce`, :593) plus the nonce. You have seen the methods named here; the format itself — why the bytes are laid out as they are, how variable-length values are encoded — is the subject of Chapter 26 (serialization), which treats this package's `get_*_struct/create_from_struct` methods as its raw material. For this chapter it is enough to know that a vertex can turn itself into bytes and back, and that its identity is the hash of those bytes.

25.6 The metadata split: static versus mutable

This is the conceptual heart of the package, and the part most worth slowing down for.

A vertex's own data — its inputs, outputs, parents, nonce — is **fixed forever** once the vertex exists, because changing it would change the hash and thus the identity (§25.5.2). But the node learns things about a vertex as the ledger grows around it. Some of that learned knowledge is itself permanent; some of it changes as the graph changes. Hathor splits these into **two separate objects**, and understanding why is understanding the package.

The problem. Consider two facts the node computes about a block:

1. Its **height** — how many blocks lie between it and genesis.
2. Whether it is currently **voided** — whether consensus has decided it lost a conflict and should not count.

These two facts have opposite natures. A block's height is determined the moment the block is created (it is one more than its block-parent's height) and can never change — a block does not move in the chain. But a block's voided status can change: a later, heavier competing chain can void it in a reorg (Chapter 10), and a still-later development could even un-void it. Storing both in one undifferentiated "metadata" blob would lose this distinction and invite bugs (code that caches height is safe; code that caches voided-status is not).

The solution: two classes.

Static metadata — computed once, never changes

`VertexStaticMetadata` (`static_metadata.py:36`) is, in its own docstring, for "vertex attributes that are not intrinsic to the vertex data, but can be calculated from only the vertex itself and its dependencies, and whose values never change." It is an abstract base (a Pydantic model — Ch 18) with one common field, `min_height` (`:48`), and two concrete subclasses:

- `BlockStaticMetadata` (`:66`) adds `height` (`:67`) and the feature-activation bit counts (`:72`, for Ch 38). `height` is computed from the block-parent's height plus one (`_calculate_height`, `:107`).
- `TransactionStaticMetadata` (`:178`) adds `closest_ancestor_block` (`:181`) — the highest-up block the transaction depends on, used by feature activation for transactions.

The genericity from §25.5.1 now pays off: because `Block` is `GenericVertex[BlockStaticMetadata]`, the expression `block.static_metadata.height` type-checks, while the same expression on a transaction is a type error — a transaction's static metadata has no `height` at all. The type variable `StaticMetadataT` is what threads each vertex kind to the right static-metadata class.

Static metadata is set once and then frozen: `set_static_metadata` (`base_transaction.py:911`) refuses to change a value already set (it asserts the new value equals the old, `:914`). Each subclass knows how to build its static metadata from storage — `init_static_metadata_from_storage` (`Block` at `block.py:378`, `Transaction` at `transaction.py:492`) — and the build is required to be "fast, ideally $O(1)$ " (`static_metadata.py:90`), because it runs for every vertex on load.

Mutable metadata — rewritten as the ledger evolves

`TransactionMetadata` (`transaction_metadata.py:40`) holds everything the node computes about a vertex that can change. Its central fields, and what they model:

```

class TransactionMetadata:
    voided_by: Optional[set[bytes]] # :45 who voids this vertex (None/empty = executed)
    accumulated_weight: int # :48 total work of this vertex + all that confirm it
    score: int # :49 the consensus metric for blocks (Ch 32)
    first_block: Optional[bytes] # :50 the first block that confirmed this tx
    spent_outputs: dict[int, list[bytes]] # :42 which tx spent each of my outputs
    conflict_with: Optional[list[bytes]] # :44 txs that conflict with this one
    validation: ValidationState # :51 how far validation has progressed (§25.7)

```

- **voided_by** (:45) is the voiding mechanism of Chapter 10. An empty or `None` set means the vertex is executed (it counts); a non-empty set means it is voided, and the set's contents say why (its own hash if it lost a conflict, a conflicting tx's hash, a special `PARTIALLY_VALIDATED_ID` marker while it is still being validated). Voiding is a mark, not a delete — that is what lets it be reversed. The class gives careful helpers (`add_voided_by` / `del_voided_by` , : 379 / :386) that maintain the "empty means `None` " memory optimization.
- **accumulated_weight** (:48) is the sum of this vertex's work and the work of everything that confirms it (Chapter 9). It grows as more vertices pile on, so it is recomputed by a graph walk (`update_accumulated_weight` , `base_transaction.py:703`).
- **score** (:49) is the metric consensus uses to choose the canonical block chain (Chapter 32). For a non-genesis transaction it defaults to 0 (`get_metadata` , `base_transaction.py:668`).
- **spent_outputs** (:42) is how the node records, without deleting anything, that output `index` of this vertex has been spent by some later transaction. `get_output_spent_by(index)` (:142) returns the spender — but only if that spender is not itself voided, which is what makes a voided spend cleanly "give the coin back" (Chapter 7 §7.6, Chapter 10).

The invariant to carry away: a vertex's own fields are immutable; its `TransactionMetadata` is the node's mutable, evolving opinion about it. Who maintains the mutable side? Consensus (Chapter 32) writes `voided_by` and `score` ; ingestion (Chapter 33) sets `first_block` and `spent_outputs` ; the accumulated-weight walk updates `accumulated_weight` . The vertex class itself only holds the metadata and offers the accessor `get_metadata()` (`base_transaction.py:646`), which lazily loads it from storage or builds a fresh one (:667) on first access. That laziness is itself an invariant worth noting: metadata is built on demand and cached on the vertex, so the same in-memory vertex always hands back the same metadata object.

	Static metadata	TransactionMetadata
Class	<code>VertexStaticMetadata</code> (+ <code>Block/Transaction</code> subclasses)	<code>TransactionMetadata</code>
File	<code>static_metadata.py</code>	<code>transaction_metadata.py</code>

	Static metadata	TransactionMetadata
Mutability	computed once, never changes	rewritten as the graph evolves
Example fields	<code>height</code> , <code>min_height</code> , <code>closest_ancestor_block</code>	<code>voided_by</code> , <code>accumulated_weight</code> , <code>score</code> , <code>first_block</code>
Who writes it	the vertex on load (<code>init_static_metadata_from_storage</code>)	consensus, ingestion, the accumulated-weight walk
Why separate	facts that can be cached and trusted forever	facts that depend on the ever-changing rest of the DAG

Keeping these apart is not pedantry. It tells every other part of the node which facts it may cache and forget about (static) and which it must re-read after the graph changes (mutable). Mistaking one for the other is a category of bug the type system and the two-class split are there to prevent.

25.7 Validation state and storage scope

Two smaller mechanisms round out the model; both are covered in depth elsewhere, so here is just enough to recognize them.

Validation state. A vertex is not validated all at once. `ValidationState` (`validation_state.py:19`) is an enum tracking how far a vertex has progressed: `INITIAL` (not validated) → `BASIC` (its own structure and graph info check out, but not its dependencies) → `FULL` (all parents and inputs reached `FULL` and full validation ran), with `CHECKPOINT` / `CHECKPOINT_FULL` for vertices whose validity is assured by a known checkpoint, and `INVALID` as the terminal failure (`:45 - :50`). The reason for a partial state is dependency order: you can check a transaction's weight before you have its parents, but you cannot check that it does not double-spend until every input transaction is itself fully validated (the docstring at `:37` explains this). The state lives on the mutable metadata (`validation`, `transaction_metadata.py:51`), and `set_validation` (`base_transaction.py:524`) keeps it in sync with the `PARTIALLY_VALIDATED_ID` marker in `voided_by` — a partially-validated vertex is "voided by" its own incompleteness until it is fully connected. Full treatment in Chapters 31 and 33.

Storage scope. Chapter 0 §0.3 mentioned that during initialization the node temporarily lets storage return data in any validation state, then narrows it. That mechanism is `TxAllowScope` (in `transaction/storage/`), and it exists precisely because of the partial-validation states above: while rebuilding its view of the ledger, the node

must be able to load not-yet-fully-valid vertices; in normal operation it should only ever hand out fully-valid ones. The scope is the switch that widens and narrows what storage will return. Full treatment with storage, Chapter 27.

25.8 How it plugs into the lifecycle

These classes are the currency the rest of Part II trades in. Tracing one vertex's life:

1. **Born or received.** A vertex is created locally (the wallet builds a transaction; a miner produces a block) or arrives from a peer as bytes, parsed by `create_from_struct` into the right subclass via `TxVersion` (§25.5.5). Its own fields are now fixed; its hash is its identity.
2. **Loaded from storage.** When the node reads a vertex back from disk, storage reconstructs the object and calls `init_static_metadata_from_storage` to attach its (immutable) static metadata, and `get_metadata` to attach its (mutable) `TransactionMetadata`. Storage and the graph-walk utilities are **Chapter 27**.
3. **Verified.** The verification pipeline checks the rules this package deliberately did not enforce — conservation of value, valid signatures (running the locking/unlocking scripts in `transaction/scripts/`), sufficient weight, structural correctness — and advances the `ValidationState`. **Chapter 31**.
4. **Consensus updates its metadata.** As the vertex is woven into the DAG, consensus computes `accumulated_weight` and `score`, decides conflicts, and writes `voided_by`. This is the mutable metadata of §25.6 being rewritten. **Chapter 32**.
5. **Ingested.** The vertex handler runs steps 3 and 4 end-to-end, sets `first_block` and `spent_outputs`, updates the indexes, and announces the change. "Data arrived" becomes "ledger changed." **Chapter 33**.

At every step the object is the same `Block` or `Transaction`; what changes is its mutable metadata, never its identity.

Recap

Concept (earlier chapter)	In code	Where
Vertex / DAG node (Ch 8)	<code>GenericVertex</code> (ABC, generic); alias <code>Vertex / BaseTransaction</code>	<code>base_transaction.py:</code> 148 , :932

Concept (earlier chapter)	In code	Where
Block (Ch 8)	<code>Block</code> → <code>MergeMinedBlock</code> , <code>PoaBlock</code>	<code>block.py:45</code> ; <code>merge_mined_block.py:31</code> ; <code>poa/poa_block.py:30</code>
Transaction (Ch 8)	<code>Transaction</code> → <code>TokenCreationTransaction</code> , <code>OnChainBlueprint</code>	<code>transaction.py:55</code> ; <code>token_creation_tx.py:35</code>
Parents — topology edge (Ch 8 §8.4)	<code>parents: list[VertexId]</code> ; <code>parents[0] = block.parent</code>	<code>base_transaction.py:204</code> ; <code>block.py:169</code>
Inputs — spending edge (Ch 7)	<code>inputs: list[TxInput]</code> , <code>(tx_id, index, data)</code>	<code>base_transaction.py:202</code> , :936
Output / coin (Ch 7)	<code>TxOutput(value, script, token_data)</code>	<code>base_transaction.py:1022</code>
Multi-token + authorities (Ch 7 §7.6)	<code>token_data</code> byte: index mask + authority bit	<code>base_transaction.py:1025</code> , :1099 , :1103
Voiding (Ch 10)	<code>TransactionMetadata.voided_by</code>	<code>transaction_metadata.py:45</code>
Accumulated weight / score (Ch 9)	<code>TransactionMetadata.accumulated_weight</code> , <code>.score</code>	<code>transaction_metadata.py:48</code> , :49
Immutable derived facts	<code>VertexStaticMetadata</code> (Block adds height)	<code>static_metadata.py:36</code> , :66
Partial validation	<code>ValidationState</code> enum	<code>validation_state.py:19</code>

The `hathor/transaction/` package is where the two great abstractions of Part I's domain track stop being diagrams and become Python you can instantiate. A vertex is a `GenericVertex` — abstract, generic over its static-metadata companion, realized as a `Block` or a `Transaction` (and four further leaves). Its `inputs` and `outputs` are the UTXO model of Chapter 7 down to the bit-packed `token_data` byte; its `parents` are the DAG topology of Chapter 8, kept rigorously separate from the inputs. Around each

vertex the node keeps two kinds of memory: static metadata it computes once and trusts forever, and mutable `TransactionMetadata` — `voided_by`, `accumulated_weight`, `score` — that consensus rewrites as the graph grows. The vertex's own fields are frozen by the hash that names it; the node's opinion about the vertex is what changes. Hold that one sentence and the rest of Part II reads cleanly. Next, Chapter 26 answers a question this chapter kept deferring: how exactly do these objects become the compact bytes that travel between nodes and onto disk?

1. `VertexId` is a type alias for `bytes` (`hathor/types.py:26`) — specifically the 32-byte double-SHA256 hash that identifies a vertex. Storing parents and inputs as `VertexId` rather than as object references keeps a vertex small and lets the storage layer load the graph lazily. ↩
2. An abstract base class (ABC) is a class you cannot instantiate directly; it defines a shared interface and marks some methods `@abstractmethod`, forcing every concrete subclass to implement them. `GenericVertex` is an ABC, so you only ever create a `Block` or a `Transaction`. Full treatment in Chapter 1. ↩
3. A generic class is parameterized by a type variable, written `Class[T]`. `GenericVertex[StaticMetadataT]` is generic over its static-metadata type, so `Block = GenericVertex[BlockStaticMetadata]` and `Transaction = GenericVertex[TransactionStaticMetadata]` stay distinct to the type-checker. Full treatment in Chapter 5. ↩
4. `__slots__` declares the fixed set of attributes instances may have, replacing each object's per-instance dictionary with a compact fixed layout. It saves memory (decisive when millions of vertices are in RAM) and prevents accidental attribute typos. It is a Python optimization, not a domain concept. ↩
5. An authority output carries no spendable value; instead its `value` field is reinterpreted as a permissions bitmask (mint and/or melt rights for a token). It is flagged by the high bit of the output's `token_data` byte. Authorities are how a token issuer retains control over a token's supply — see Chapter 7 §7.6. ↩
6. The sighash ("signature hash") is the hash of a transaction's body that a signer actually signs. Each input's `data` (unlocking script) is blanked while computing it, because you cannot sign over the signature you are in the middle of producing. `Transaction` caches it (`transaction.py:93`) because signing many inputs re-serializes the same body repeatedly. ↩
7. A reorg (reorganization) is when the node switches its canonical block chain to a different, heavier one, voiding the blocks of the chain it abandons. Because voiding is a reversible mark on mutable metadata (`voided_by`), not a deletion, a reorg can flip a block from executed to voided without destroying its data. Full treatment in Chapter 32. ↩